Disaster Relief Resource Management System

Course: Advanced Database Management System Practical

Database: MySQL

Project Description

The Disaster Relief Resource Management System (DRRMS) will be developed as a database project for storing and managing disaster events, shelters, donors, donated resources, relief requests, volunteers and resource distribution. Self-created dataset will be used to complete database design, populated and tested using the concepts covered in the ADBMS practical.

Objectives

1. To enable the design of a structured dataset using proper data modelling techniques.

2. To develop practical skills in SQL for database creation, manipulation and querying.

3. To enhance analytical thinking through multi-phase database development.

Phase 1 - Data Modelling and Dataset Design

1. Dataset: A small DRRMS dataset containing disaster events, shelters, donors, resources, requests, volunteers and distribution records. The same data will be used in all later phases.

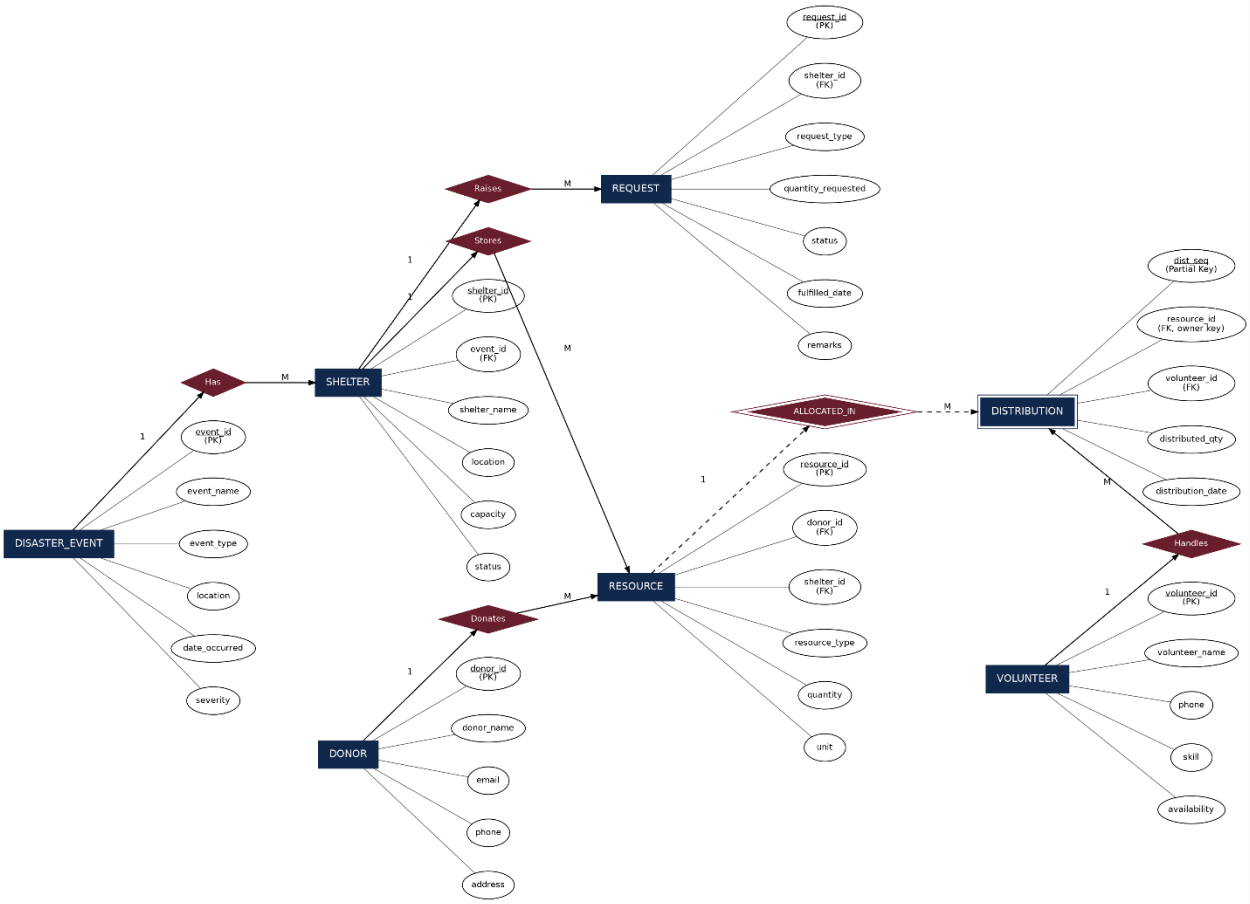
2. Entity and attribute identification: DISASTER_EVENT, SHELTER, DONOR, RESOURCE, REQUEST, VOLUNTEER and DISTRIBUTION as the main entities, with attributes required to store their respective details.

3. ER modelling: DISASTER_EVENT-SHELTER, DONOR-RESOURCE, SHELTER-RESOURCE, SHELTER-REQUEST, RESOURCE-DISTRIBUTION and VOLUNTEER-DISTRIBUTION will be modelled mainly as 1:M relationships.

4. Relational schema: Convert the ER model into tables. Primary keys will identify records and foreign keys will connect related tables.

5. Normalization: donor, shelter, request and distribution details will be stored separately so that repeated data is reduced and the schema remains in 3NF.

ER Diagram



Tables and Data Types

Table	Fields with Data Type	Relationship
DISASTER_EVENT	event_id INT PK AUTO_INCREMENT event_type VARCHAR(40) location VARCHAR(60) start_date DATE severity VARCHAR(15)	DISASTER_EVENT 1:M SHELTER
DONOR	donor_id INT PK AUTO_INCREMENT donor_name VARCHAR(60) NOT NULL phone VARCHAR(15) email VARCHAR(80) UNIQUE donor_type VARCHAR(20)	DONOR 1:M RESOURCE
VOLUNTEER	volunteer_id INT PK AUTO_INCREMENT volunteer_name VARCHAR(60) NOT NULL phone VARCHAR(15) skill VARCHAR(40)	VOLUNTEER 1:M DISTRIBUTION

Table	Fields with Data Type	Relationship
	availability_status VARCHAR(20)	
SHELTER	shelter_id INT PK AUTO_INCREMENT event_id INT FK location VARCHAR(60) capacity INT current_occupancy INT	FK event_id -> DISASTER_EVENT SHELTER 1:M RESOURCE SHELTER 1:M REQUEST
RESOURCE	resource_id BIGINT PK AUTO_INCREMENT donor_id INT FK shelter_id INT FK resource_type VARCHAR(30) quantity DECIMAL(10,2) unit VARCHAR(15)	FK donor_id -> DONOR FK shelter_id -> SHELTER RESOURCE 1:M DISTRIBUTION
REQUEST	request_id INT PK AUTO_INCREMENT shelter_id INT FK resource_type VARCHAR(30) quantity_requested DECIMAL(10,2) request_status VARCHAR(20) request_date DATE fulfilled_date DATE NULL	FK shelter_id -> SHELTER
DISTRIBUTION	distribution_id BIGINT PK AUTO_INCREMENT resource_id BIGINT FK volunteer_id INT FK distributed_qty DECIMAL(10,2) distributed_at DATETIME	FK resource_id -> RESOURCE FK volunteer_id -> VOLUNTEER

Relational Schema: DISASTER_EVENT(event_id) -> SHELTER(event_id); DONOR(donor_id) -> RESOURCE(donor_id); SHELTER(shelter_id) -> RESOURCE(shelter_id), REQUEST(shelter_id); RESOURCE(resource_id) -> DISTRIBUTION(resource_id); VOLUNTEER(volunteer_id) -> DISTRIBUTION(volunteer_id).

Phase 2 - Database Creation and Data Preparation

1. DDL: Create DRRMS_DB and all seven tables using CREATE DATABASE and CREATE TABLE. PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL and CHECK constraints will be defined with the tables.

2. DML: Enter sample data using INSERT. UPDATE will be used when request status or resource quantity changes, while DELETE will be tested only where foreign-key relations are not affected.

3. Data cleaning and constraints: duplicate donor emails and shelter locations will be prevented, required IDs will not be NULL, and invalid negative quantity or capacity values will be restricted.

4. NULL handling: fulfilled_date will stay NULL while a request is pending and remarks can stay NULL when there is no comment. IS NULL / IS NOT NULL will be used to find such records.

5. Basic queries: SELECT, WHERE and ORDER BY will be used to display active shelters, pending requests, resources by type, volunteers by skill and distributions by date.

6. Aggregate functions: COUNT() will calculate requests per shelter, AVG() will calculate average resource quantity per type and SUM() will calculate total resources distributed.

Phase 3 - Data Querying and Analytical Operations

1. Joins: join DONOR + RESOURCE for donation details, SHELTER + REQUEST + DISASTER_EVENT for relief reports, and RESOURCE + DISTRIBUTION for allocation history.

2. Nested queries and subqueries: find shelters whose request count is above the average request count and resources whose distributed quantity is above the overall average.

3. View: create Shelter_Resource_Summary to display shelter name, disaster event, resource type, quantity and distributed quantity through one reusable query.

4. Query execution and optimization: use EXPLAIN on the shelter-request-event join. Indexes will be created on frequently searched columns such as RESOURCE.shelter_id, REQUEST.shelter_id and DISTRIBUTION.resource_id and the query plan will be checked again.

5. Function: create GetShelterRequestCount(shelter_id). It will receive a shelter ID, count the matching records in REQUEST and return the total requests raised by that shelter.

6. Procedure: create AddDistribution(resource_id, volunteer_id, distributed_qty). It will accept the distribution details, insert a new DISTRIBUTION record and reduce the available quantity in RESOURCE.

Functional Modules

Function / Task	Tables Used	How I will do it
Register donor and resource	DONOR, RESOURCE	I will add the donor first and then store the donated resource using donor_id as the foreign key.
Add and view shelters	DISASTER_EVENT, SHELTER	I will store shelter location, capacity and current occupancy against the linked disaster event.
Raise a request	SHELTER, REQUEST	I will record the resource type and quantity requested by a shelter and update its status as it is fulfilled.

Function / Task	Tables Used	How I will do it
Allocate distribution	RESOURCE, VOLUNTEER, DISTRIBUTION	I will select an available resource and volunteer, create the distribution record and reduce the resource quantity in one transaction.
Track disaster events	DISASTER_EVENT, SHELTER	I will store event type, location, start date and severity and link shelters opened for that event.
Generate summary reports	DONOR, RESOURCE, SHELTER, REQUEST, DISTRIBUTION	I will use joins, aggregates and the Shelter_Resource_Summary view to prepare donation, request and distribution summaries.

Phase 4 - Data Integrity and Transaction Handling

1. Transaction handling: when I allocate a resource to a distribution, I will start one transaction, insert the DISTRIBUTION record and reduce RESOURCE.quantity. A SAVEPOINT will be used before the final update. COMMIT will be used if both operations succeed and ROLLBACK if an error occurs.

2. Atomicity: the distribution insertion and resource-quantity update will be treated as one unit. If either operation fails, the complete transaction will be rolled back so the database does not remain partly updated.

3. Concurrency case: before confirming a distribution I will check that the selected resource still has sufficient quantity inside the transaction. This will help prevent the same resource from being over-allocated to two distributions at the same time.

4. Integrity constraints: foreign keys will ensure that SHELTER uses an existing DISASTER_EVENT, RESOURCE uses an existing DONOR and SHELTER, REQUEST uses an existing SHELTER, and DISTRIBUTION uses an existing RESOURCE and VOLUNTEER.

5. Final validation: check for missing references, confirm that fulfilled requests match distributed quantities, and run the basic queries, joins, subqueries, view, function, procedure and transaction cases to verify the database.

Outcome of the Project

1. A normalized relational database for disaster relief resource records up to 3NF.

2. Practical implementation of DDL, DML, constraints, basic queries, aggregates, joins, subqueries, views, functions/procedures and transactions.

3. Disaster event, shelter, donation, request and distribution information can be stored and retrieved through related tables.

4. The project will give hands-on understanding of data modelling, SQL querying, integrity and transaction handling using one complete database case study.

Normalization Analysis (1NF, 2NF, 3NF, BCNF) and Relational Schema

This section extends Phase 1 of the DRRMS proposal by verifying the normal form of each table and restating the complete relational schema with primary and foreign keys.

1. Normalization Analysis

Every table below has a single-column primary key (no composite keys are used anywhere in the design), so no non-key attribute can ever depend on only part of the key. This means partial dependency is impossible by construction, and 2NF is automatically satisfied wherever 1NF holds. Each table is therefore checked for atomicity (1NF), transitive dependency (3NF), and whether every determinant is a candidate key (BCNF).

DISASTER_EVENT (event_id PK)

1NF	All attributes (event_type, location, start_date, severity) hold single, atomic values. No repeating groups or multi-valued fields. Satisfied.
2NF	Primary key is the single column event_id, so no partial dependency is possible. Satisfied.
3NF	event_type, location, start_date and severity each depend only on event_id; none of them determines another. No transitive dependency. Satisfied.
BCNF	The only determinant is event_id, which is the candidate key. Satisfied.

DONOR (donor_id PK)

1NF	donor_name, phone, email and donor_type are atomic (a donor is not allowed multiple emails/phones in one row). Satisfied.
2NF	Single-column key donor_id; no partial dependency possible. Satisfied.
3NF	email is UNIQUE and could act as an alternate candidate key, but it does not determine donor_type or phone in a way that removes them from donor_id. No non-key attribute determines another non-key attribute. Satisfied.
BCNF	Determinants present: donor_id (PK) and email (candidate key, since UNIQUE + assumed NOT NULL). Both are candidate keys. Satisfied.

VOLUNTEER (volunteer_id PK)

1NF	volunteer_name, phone, skill, availability_status are atomic single values per volunteer. Satisfied.
2NF	Single-column key volunteer_id; no partial dependency possible. Satisfied.
3NF	No attribute among phone, skill or availability_status determines another. All depend only on volunteer_id. Satisfied.
BCNF	Only determinant is volunteer_id, the candidate key. Satisfied.

SHELTER (shelter_id PK)

1NF	event_id, location, capacity, current_occupancy are atomic. Satisfied.
2NF	Single-column key shelter_id; no partial dependency possible. Satisfied.
3NF	capacity and current_occupancy depend on shelter_id, not on each other or on event_id (a numeric capacity value is not functionally determined by the disaster event). No transitive dependency. Satisfied.
BCNF	Only determinant is shelter_id, the candidate key. event_id is a foreign key reference, not a determinant of other attributes here. Satisfied.

RESOURCE (resource_id PK)

1NF	donor_id, shelter_id, resource_type, quantity, unit are atomic. Satisfied.
2NF	Single-column key resource_id; no partial dependency possible. Satisfied.
3NF	quantity and unit depend on resource_id. Design note: if resource_type were treated as always fixing one unit (e.g. "Water" is always recorded in litres), that would be a transitive dependency resource_type -> unit and would break 3NF. As modelled here, unit is stored per resource_id (not derived from resource_type alone), so the table remains in 3NF; this is flagged as a point worth discussing with the guide.
BCNF	With unit treated as dependent on resource_id rather than resource_type, resource_id remains the only determinant and the candidate key. Satisfied (subject to the note above).

REQUEST (request_id PK)

1NF	shelter_id, resource_type, quantity_requested, request_status, request_date, fulfilled_date are atomic. fulfilled_date is allowed to be NULL, which does not violate 1NF. Satisfied.
2NF	Single-column key request_id; no partial dependency possible. Satisfied.
3NF	request_status and fulfilled_date depend on request_id, not on each other or on shelter_id/resource_type. No transitive dependency. Satisfied.
BCNF	Only determinant is request_id, the candidate key. Satisfied.

DISTRIBUTION (distribution_id PK)

1NF	resource_id, volunteer_id, distributed_qty, distributed_at are atomic. Satisfied.
2NF	Single-column key distribution_id; no partial dependency possible. Satisfied.
3NF	distributed_qty and distributed_at depend only on distribution_id, not on resource_id or volunteer_id individually. No transitive dependency. Satisfied.
BCNF	Only determinant is distribution_id, the candidate key. Satisfied.

Conclusion: All seven tables (DISASTER_EVENT, DONOR, VOLUNTEER, SHELTER, RESOURCE, REQUEST, DISTRIBUTION) are already in Boyce-Codd Normal Form (BCNF), which is the strictest of the four levels checked. Because every table uses a single, non-composite primary key and no non-key attribute determines another non-key attribute, the schema satisfies 1NF, 2NF, 3NF and BCNF simultaneously, matching the 3NF target stated in the Phase 1 proposal.

2. Relational Schema

Underlined attributes are primary keys; attributes marked FK are foreign keys referencing the named parent table and column.

DISASTER_EVENT (event_id, event_type, location, start_date, severity)

DONOR (donor_id, donor_name, phone, email, donor_type)

VOLUNTEER (volunteer_id, volunteer_name, phone, skill, availability_status)

SHELTER (shelter_id, event_id [FK → DISASTER_EVENT.event_id], location, capacity, current_occupancy)

RESOURCE (resource_id, donor_id [FK → DONOR.donor_id], shelter_id [FK → SHELTER.shelter_id], resource_type, quantity, unit)

REQUEST (request_id, shelter_id [FK → SHELTER.shelter_id], resource_type, quantity_requested, request_status, request_date, fulfilled_date)

DISTRIBUTION (distribution_id, resource_id [FK → RESOURCE.resource_id], volunteer_id [FK → VOLUNTEER.volunteer_id], distributed_qty, distributed_at)

Referential (Foreign Key) Relationships Summary

Parent Table	Child Table	Relationship	Foreign Key
DISASTER_EVENT	SHELTER	1 : M	SHELTER.event_id → DISASTER_EVENT.event_id
DONOR	RESOURCE	1 : M	RESOURCE.donor_id → DONOR.donor_id
SHELTER	RESOURCE	1 : M	RESOURCE.shelter_id → SHELTER.shelter_id
SHELTER	REQUEST	1 : M	REQUEST.shelter_id → SHELTER.shelter_id
RESOURCE	DISTRIBUTION	1 : M	DISTRIBUTION.resource_id → RESOURCE.resource_id
VOLUNTEER	DISTRIBUTION	1 : M	DISTRIBUTION.volunteer_id → VOLUNTEER.volunteer_id